



DACEFyN

Departamento Académico de Ciencias
Exactas, Físicas y Naturales

ASIGNATURA: PROGRAMACIÓN III

TRABAJO PRACTICO Nº 1.1

UNIDAD 1: FUNDAMENTOS Y DESARROLLO DE POO

Sistema de Gestión de Biblioteca Universitaria

Alumno/a: BRIZUELA MARCO EZEQUIEL

OBJETIVO EDUCATIVO

Consolidar los fundamentos de la Programación Orientada a Objetos aplicando encapsulamiento, estructuras de datos (Collections Framework) y manejo de excepciones en un contexto realista de gestión de préstamos bibliotecarios.

1. CONTEXTO

La biblioteca de la facultad necesita modernizar su sistema de préstamos. Se requiere una aplicación de consola que permita gestionar libros, estudiantes y préstamos, aplicando buenas prácticas de POO y manejo de errores.

2. REQUISITOS FUNCIONALES

2.1 Modelado de Entidades (POO Básica)

- ❑ Clase Libro: ISBN (String), título (String), autor (String), año (int), disponible (boolean)
- ❑ Clase Estudiante: legajo (String), nombre (String), carrera (String), email (String)
- ❑ Clase Préstamo: libro (Libro), estudiante (Estudiante), fechaPréstamo (LocalDate), fechaDevolucion (LocalDate)

2.2 Estructuras de Datos (Java Collections)

- ❑ Catálogo de libros: ArrayList<Libro>
- ❑ Registro de estudiantes: HashMap<String, Estudiante> (clave: legajo)
- ❑ Préstamos activos: HashSet<Préstamo> (evitar duplicados mediante equals() y hashCode())

2.3 Manejo de Excepciones Personalizadas

- ❑ LibroNoDisponibleException: intento de préstamo sin stock
- ❑ EstudianteNoEncontradoException: legajo inexistente
- ❑ LimitePréstamosExcedidoException: máximo 3 libros por estudiante

2.4 Funcionalidades Principales

- ❑ Registrar préstamo: validar disponibilidad, límite de estudiante, registrar fecha
- ❑ Registrar devolución: calcular posible multa, liberar libro
- ❑ Buscar libros: búsqueda parcial por título (case-insensitive)
- ❑ Listar préstamos: por estudiante específico

2.5 Recursividad

- ❑ Método calcularMulta(int diasRetraso, double valorLibro): 1% por día de retraso, máximo 30 días calculables recursivamente
- ❑ Análisis de la pila de llamadas para 30 iteraciones

3. RESTRICCIONES TECNICAS

0. Todos los atributos privados con getters/setters
1. Constructores parametrizados y por defecto
2. Método toString() informativo en todas las clases

4. ENTREGABLES

1. Código fuente en estructura de paquetes: model, exception, service, ui
2. Diagrama de clases UML (puede ser imagen o descripción textual detallada)
3. Clase Main con casos de prueba que demuestren:
 3. Creación de al menos 5 libros y 3 estudiantes
 4. Manejo exitoso de prestamos
 5. Captura y manejo de las 3 excepciones personalizadas
 6. Cálculo de multa con retraso de 15 días

5. CRITERIOS DE EVALUACION

Aspecto	Peso
Modelado correcto de clases	25%
Uso apropiado de Collections	20%
Implementacion de excepciones	20%
Funcionalidad de negocio	20%
Implementación recursiva correcta	10%
Documentacion JavaDoc basica	5%